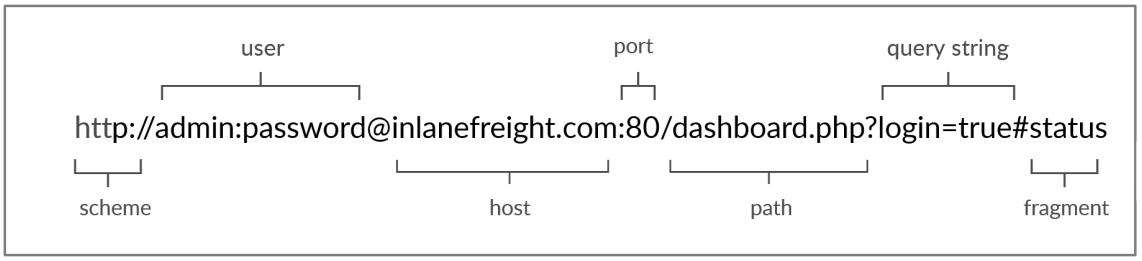
HyperText Transfer Protocol (HTTP)

HTTP- poziom aplikacji iso/osi
jak jest złożony adres url



źródło: https://cdn.services-k8s.prod.aws.htb.systems/content/modules/35/url_structure.png

`http:// https://` - używa się di identyfikacji jaki protokół klient prosi
`admin:password@` - informacje o userze
`inlanefreight.com` - nazwa hosta
`:80` - port
`/dashboard.php` - ścieżka do pliki strony
`?login=true` - kolejka oczekiwania do logowania
Jak wygląda flow komunikacji użytkownik strona serwer.

Użytkownik wpisuje adres strony w przeglądarce.

Komputer pyta serwer DNS o adres IP przypisany do tej domeny.

Przeglądarka łączy się z tym adresem IP i otwiera bezpieczne połączenie.

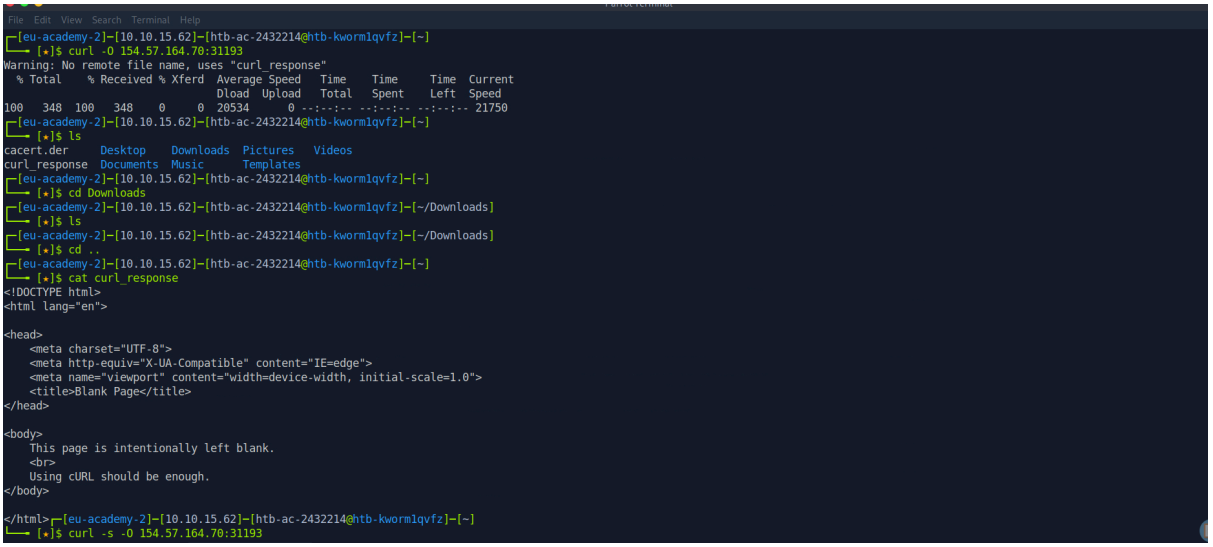
Przeglądarka wysyła żądanie pobrania strony (np. metodą GET)

Serwer odsyła kod strony (HTML, obrazy, style).

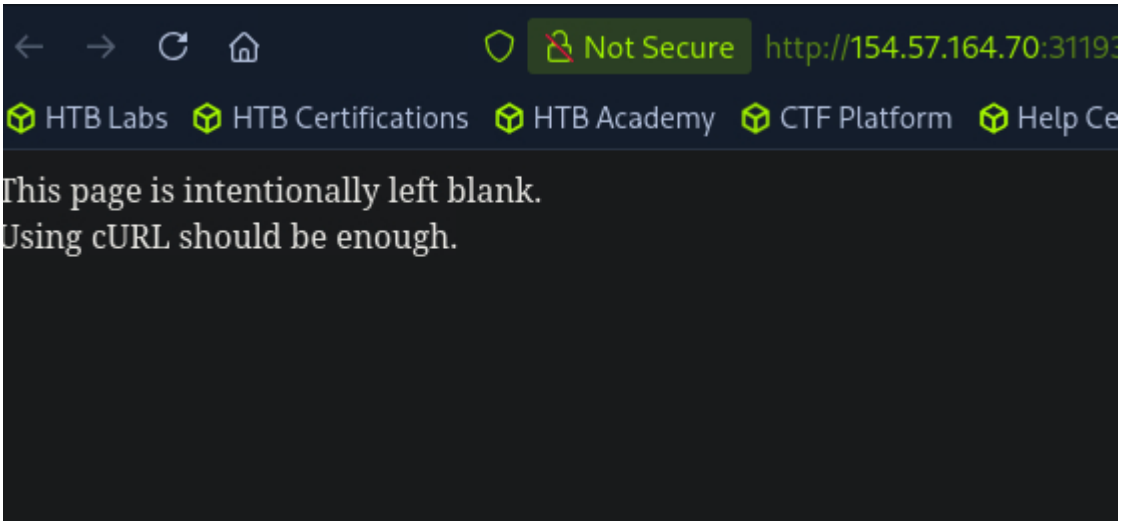
Przeglądarka składa te elementy i pokazuje je użytkownikowi

Pobieranie pliku

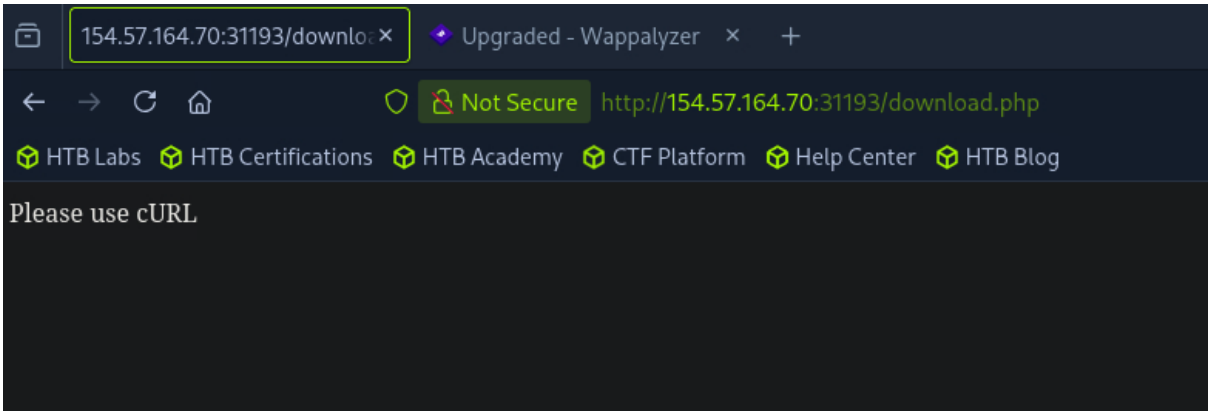
curl -s -O adres_lub_ip_pliku/nazwa_pliku: pobranie pliku z strony



na początku wykorzsytałem komende curl -O 154.57.164.70:31193 by pobrać plik ale jak widać pobrany plik był surową stroną więc nie wykonam zadania. Po tym wszedłem na stronę firefox i zobaczyłem zwykłą strona taka jak surowa tyko że wykonana z poziomu serwer na polecenie usera. zdjęcie poniżej



wpisanie w przeglądarce pliku na pobranie wyświetla takie coś XD



sprawdziłem polecenie i przepisałem dokładnie komendę plus plik który trzeba pobrać:
curl -s -O adres_lub_ip_pliku/nazwa_pliku

pobrany plik otworzyłem za pomocą komendy cat i zrobiłem pokój.

lecimy dalej

HTTPS

Różnica między http a https jest taka, że zwartość protokołu http idzie przeczytać w wiresharku ponieważ nie jest szyfrowany. To znaczy że po przechwyceniu przepływu informacji między serwerami nie można odczytać hasła usera do danej strony internetowej.

Jak wygląda komunikacja między userem a serwerem by zobaczyć stronę na protokole HTTPS?

Krok po kroku: Od HTTP do bezpiecznego HTTPS

1.Próba połączenia: Przeglądarka wysyła pierwsze zapytanie na port 80 (niezaszyfrowane HTTP).

2.Przekierowanie (Redirect): Serwer zwraca kod 301 Moved Permanently i wskazuje bezpieczny port 443.

3.Powitanie klienta (Client Hello): Przeglądarka wysyła swoje specyfikacje (np. wersje TLS, obsługiwane szyfry).

3.Powitanie serwera (Server Hello): Serwer odpowiada, wybierając parametry połączenia i przesyłając swój certyfikat SSL/TLS.

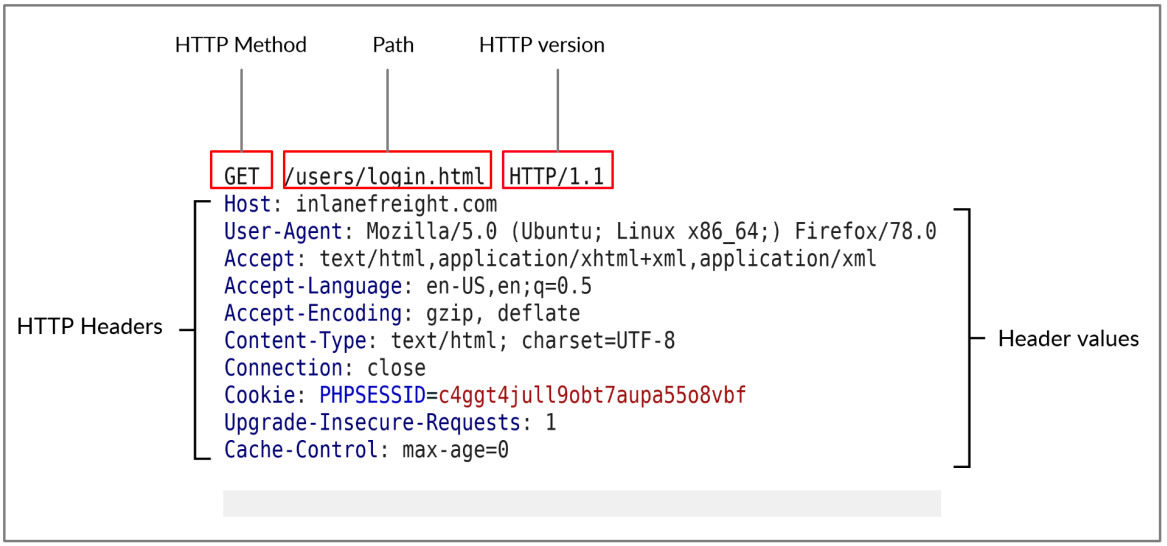
4.Weryfikacja i wymiana kluczy: Przeglądarka sprawdza ważność certyfikatu i obie strony bezpiecznie ustalają wspólny klucz sesyjny.

5.Szyfrowany uścisk dłoni (Handshake): Obie strony potwierdzają testową wiadomością, że szyfrowanie działa poprawnie.

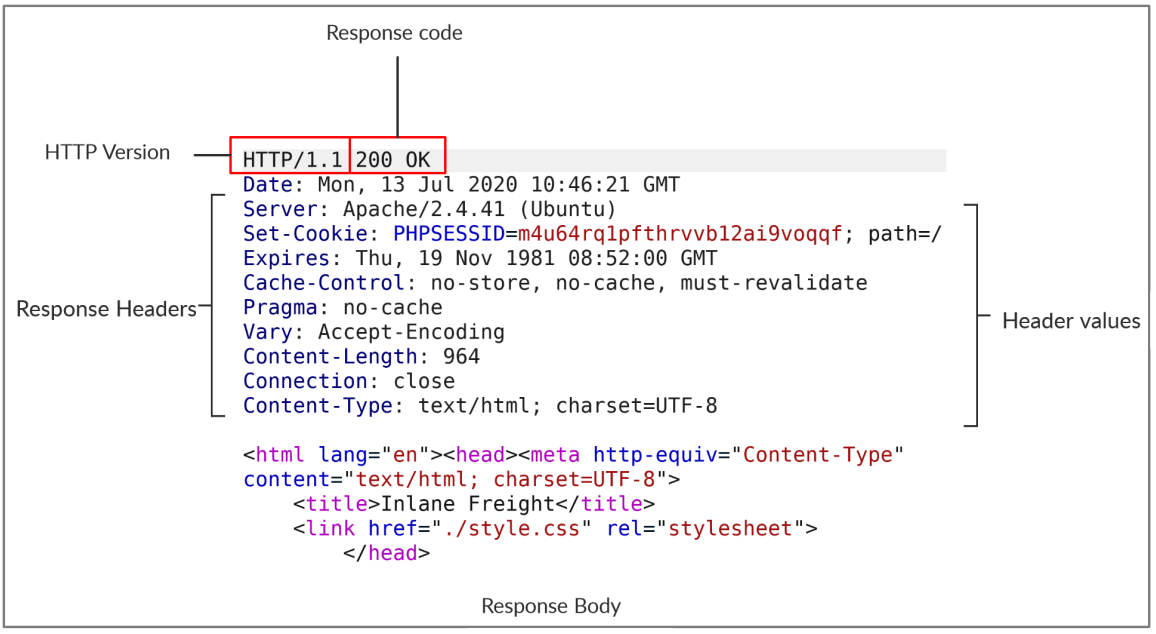
6.Bezpieczna komunikacja: Dalsza wymiana danych (HTTP) odbywa się już wewnątrz zaszyfrowanego tunelu.

Żeby użyć poprawnie curl pomijając certyfikat trzeba użyć -k i zobaczysz surową stronę.

Zapytanie - jest wysyłane od użytkownika, wysyła niezbędne dane jaka domena jaki certyfikat i inne dane potrzebne do serwera http
odpowiedz - to jest wysyłane od serwera i przetworzone przez serwer
Zapytanie:



Źródło: https://cdn.services-k8s.prod.aws.htb.systems/content/modules/35/raw_request.png
Odpowiedź serwera:



Źródło:
https://cdn.services-k8s.prod.aws.htb.systems/content/modules/35/raw_response.png

Gdy użyjemy -v przy komedzie curl zonbaczymycała ścieżkę zapytania i odpowiedzi. Tak to wygląda:

```
OtwartyHaker@htb[/htb]$ curl inlanefreight.com -v

*   Trying SERVER_IP:80...
* TCP_NODELAY set
* Connected to inlanefreight.com (SERVER_IP) port 80 (#0)
> GET / HTTP/1.1
> Host: inlanefreight.com
> User-Agent: curl/7.65.3
> Accept: */*
> Connection: close
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 401 Unauthorized
< Date: Tue, 21 Jul 2020 05:20:15 GMT
< Server: Apache/X.Y.ZZ (Ubuntu)
< WWW-Authenticate: Basic realm="Restricted Content"
< Content-Length: 464
< Content-Type: text/html; charset=iso-8859-1
<
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>

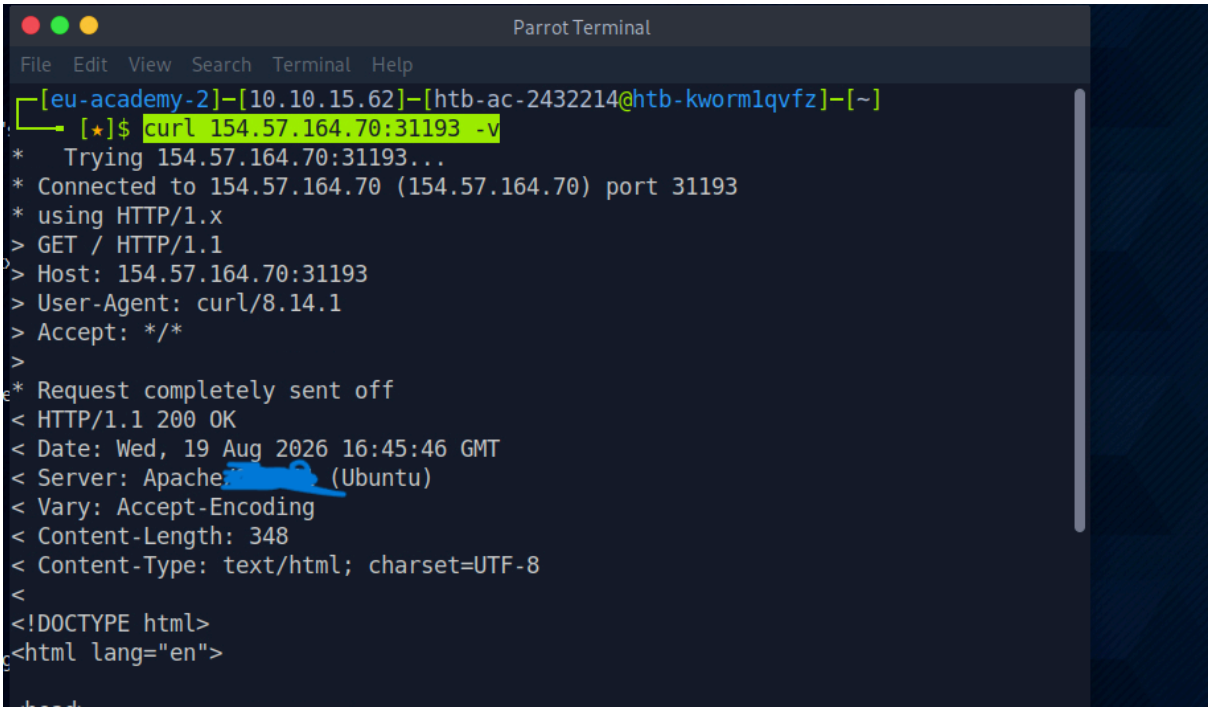
...SNIP...
```

Zobaczysz więcej przy użyciu `-vvv`

Wykonanie i zodbycie dziwnej flagi dzięki komendzie `curl inlanefreight.com -v`

by znaleźć wesje Apacha

```
curl 154.57.164.70:31193 -v
```


Parrot Terminal
File Edit View Search Terminal Help
[eu-academy-2]-[10.10.15.62]-[htb-ac-2432214@htb-kworm1qvfz]-[~]
[★]$ curl 154.57.164.70:31193 -v
* Trying 154.57.164.70:31193...
* Connected to 154.57.164.70 (154.57.164.70) port 31193
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 154.57.164.70:31193
> User-Agent: curl/8.14.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Wed, 19 Aug 2026 16:45:46 GMT
< Server: Apache (Ubuntu)
< Vary: Accept-Encoding
< Content-Length: 348
< Content-Type: text/html; charset=UTF-8
<
<!DOCTYPE html>
<html lang="en">
<head>

Generalne nagłówki mają datę wysłania zapytania
connection - stan połączenia wysłanego zapytania

Entity headers/ nagłówki jednostek -

Skrócona notatka do zapamiętania:
Content-Type - CO wysyłam?
charset - JAK odczytać tekst?
boundary - GDZIE są granice części?
Content-Length - ILE tego jest?
Content-Encoding - JAK zostało zakodowane/skompresowane?

Request headers nagłski zpaytań

Header	Example	Description
`Host`	`Host: www.inlanefreight.com`	Co jest źródłem hosta
`User-Agent`	`User-Agent: curl/7.77.0`	Informacje z czego korzysta klient
`Referer`	`Referer: http://www.inlanefreight.com/`	Informacje gdzie idzie aktualnie zapytanie
`Accept`	`Accept: */*`	Wyjaśnia jakie media klient rozumie

`Cookie`	`Cookie: PHPSESSID=b4e4fbd93540`	Wartość ciasteczek.POd jaką zmienna są zapisane informacje o ciasteczkach. Id ciasteczek
`Authorization`	`Authorization: BASIC cGFzc3dvcmQK`	Jedna z metod identyfikacji klienta. Zwrócenie unikalne tokenu dla danego klienta.

Response Headers/ Nagłówek odpowiedzi serwera

Header	Example	Description
`Server`	`Server: Apache/2.2.14 (Win32)`	Info o serwerze
`Set-Cookie`	`Set-Cookie: PHPSESSID=b4e4fbd93540`	Zapisanie ID ciasteczek danego klienta na przyszłość.
`WWW-Authenticate`	`WWW-Authenticate: BASIC realm="localhost"`	Rodzaj autoryzacji klienta do dostępu do strony.

Security Headers/ Nagłówek Zabezpieczeń

`Content-Security-Policy`	`Content-Security-Policy: script-src 'self'`	Ten nagłówek ma instrukcje do akceptacji zaufanych stron co przeciwdziała XSS
`Strict-Transport-Security`	`Strict-Transport-Security: max-age=31536000`	Zabezpiecza przeglądarkę by informacje wysyłane były w obrębie HTTPS co będzie przeciwdziałało SNifowaniu i i przejecie hasła i loginu za pomocą Wiresharka poniewaz będzie to szyforwane.
`Referrer-Policy`	`Referrer-Policy: origin`	Określa, czy przeglądarka powinna uwzględniać wartość wskazaną w nagłówku Referer. Może to pomóc w uniknięciu ujawnienia poufnych adresów URL oraz informacji podczas przeglądania witryny

Link do więcej nagłówków:
https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers

curl -I - wyświetla tylko dostępne nagłówki

```
OtwartyHaker@htb[/htb]$ curl -I https://www.inlanefreight.com

Host: www.inlanefreight.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_14_5)
AppleWebKit/605.1.15 (KHTML, like Gecko)
Cookie: cookie1=298zf09hf012fh2; cookie2=u32t4o3tb3gg4
Accept: text/plain
Referer: https://www.inlanefreight.com/
Authorization: BASIC cGFzc3dvcmQK

Date: Sun, 06 Aug 2020 08:49:37 GMT
Connection: keep-alive
Content-Length: 26012
Content-Type: text/html; charset=ISO-8859-4
Content-Encoding: gzip
Server: Apache/2.2.14 (Win32)
```

```
Set-Cookie: name1=value1,name2=value2; Expires=Wed, 09 Jun
2021 10:18:14 GMT
WWW-Authenticate: BASIC realm="localhost"
Content-Security-Policy: script-src 'self'
Strict-Transport-Security: max-age=31536000
Referrer-Policy: origin
```

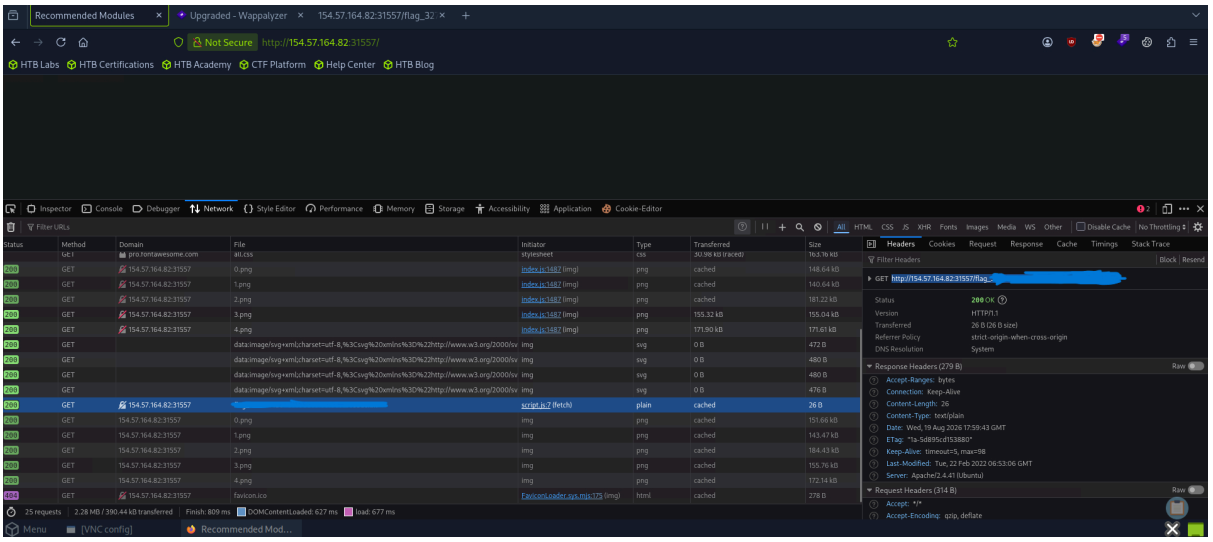
curl -i - wyświetla body i nagłówki
curl -A - zobaczysz User-Agent

```
OtwartyHaker@htb[/htb]$ curl https://www.inlanefreight.com -A
'Mozilla/5.0'
```

```
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
...SNIP...
```

W wyszukiwarce po kliknięciu F12, w sekcji Network zobaczysz informacje o zapytaniach itp.

We fladze trzeba było wykorzystać F12 i znaleźć plik flag_ i wejść w niego i wpisać zawartość flagi.

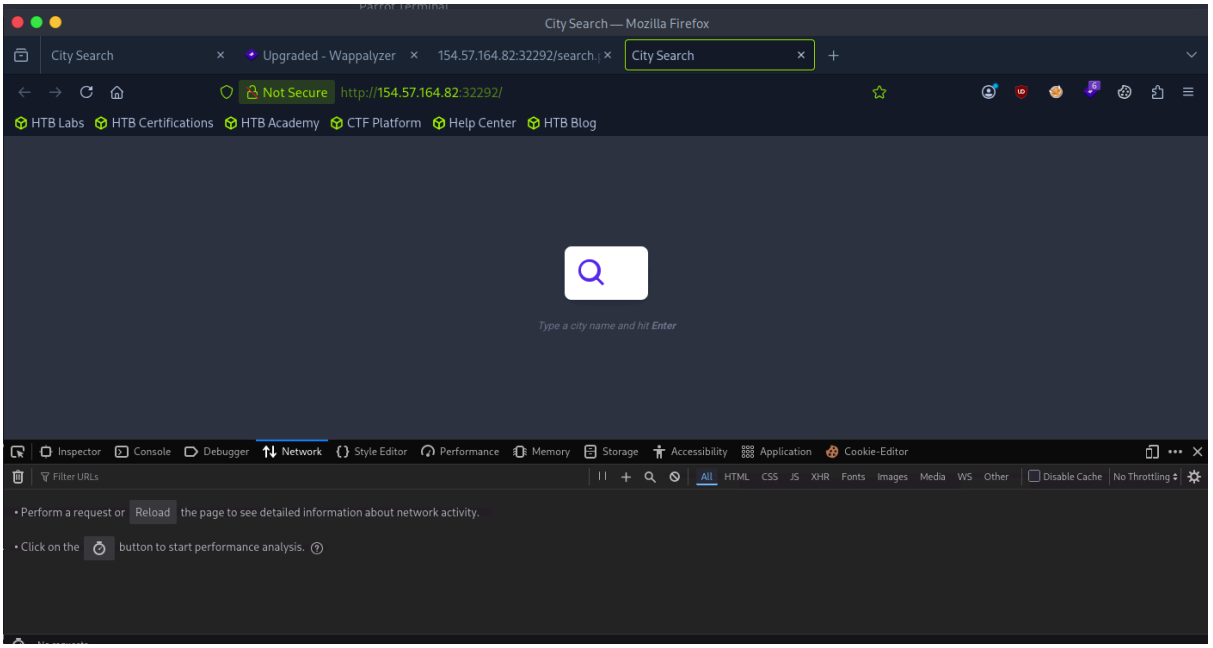


wszedłem w to i znalazłem zawartość w przeglądarce.

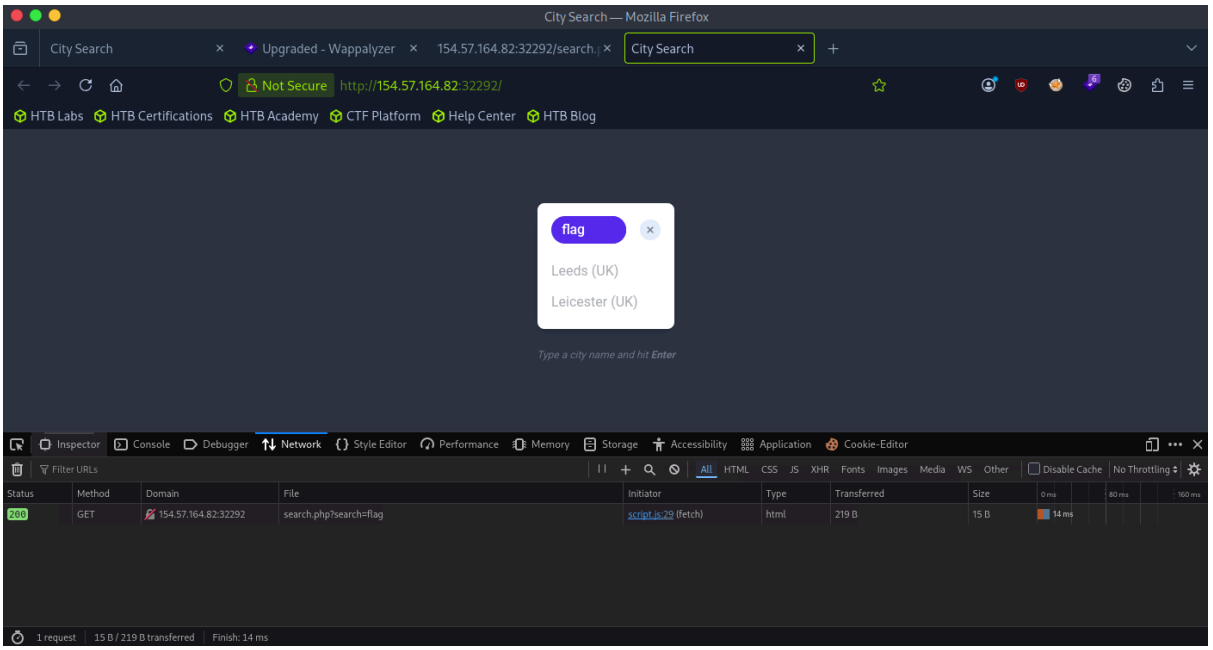
Tak brzmi zadanie do wykonania:

The exercise above seems to be broken, as it returns incorrect results. Use the browser devtools to see what is the request it is sending when we search, and use cURL to search for 'flag' and obtain the flag.

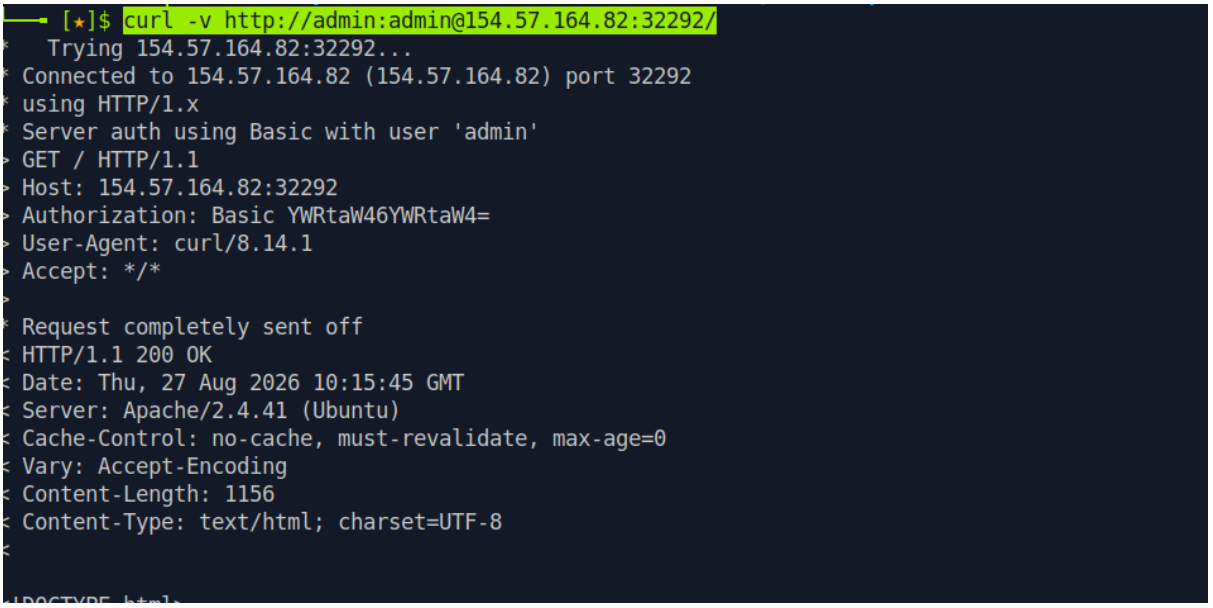
Więc wpierw wszedłem do przeglądarki wpisałem adres z portem.
Odpaliłem narzędzia deweloperskie przeglądarki i zalogowałem się admin:admin do okienka.



Wpisałem flag i wyskoczył mi adres do tego wyszukiwania. Po czym go skopiowałem.



Następnie wkleiłem adres z zalogowanym adminem w konsoli :
curl -v http://admin:admin@154.57.164.82:32292/

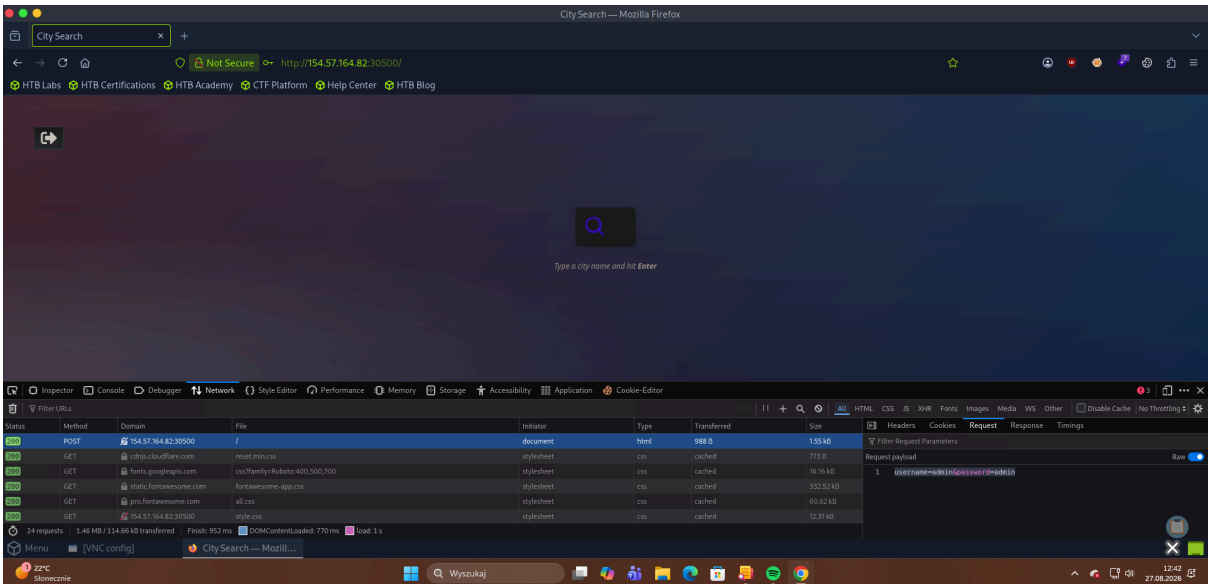


Widać informacje:
> Authorization: Basic YWRtaW46YWRtaW4=
To jest zapis logowania tylko że zaszyfrowany admin:admin
Następnie wykonał zadanie i wkleiłem tego linka na podstawie tego co było u góry.
curl 'http://154.57.164.82:32292/search.php?search=flag' -H 'Authorization: Basic YWRtaW46YWRtaW4=' i zobaczyłem flagę której nie zobaczycie tu ;)

Mamy takie zadanie:

Obtain a session cookie through a valid login, and then use the cookie with cURL to search for the flag through a JSON POST request to '/search.php'

Zalogowałem sie z odpalonymi narzędziami w przeglądarce



Znalazłem zapytanie które jest wysyłane do serwera ip i port, ale też hasło z loginem.
teraz używamy komendy curl z prefiksem wysyłania POST z hasłem.

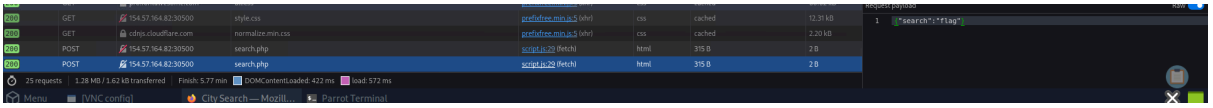
```
[eu-academy-2]─[10.10.14.186]─[htb-ac-2432214@htb-huukytc1r5]─[~]
└──╼ [★]$ curl -X POST -d'username=admin&password=admin' http://154.57.164.82:30500/

<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <title>City Search</title>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/meyer-reset/2.0/reset.min.css">
    <link rel='stylesheet' href='https://fonts.googleapis.com/css?family=Roboto:400,500,700'>
    <link rel='stylesheet' href='https://static.fontawesome.com/css/fontawesome-app.css'>
    <link rel='stylesheet' href='https://pro.fontawesome.com/releases/v5.2.0/css/all.css'>
    <link rel="stylesheet" href="./style.css">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/normalize/5.0.0/normalize.min.css">
    <script src="https://cdnjs.cloudflare.com/ajax/libs/prefixfree/1.0.7/prefixfree.min.js"></script>
</head>

<body>
        <div class="icon" style="position: absolute; top: 50px; left: 50px;">
      <a href="/?logout" class="btn btn-success"><i class="fab fas fa-sign-out-alt fa-2x"></i></a>
    </div>
    <div>
        <div class="search">
            <div class="bar">
                <div class="icon">
                    <i></i>
                </div>
            </div>
            <form>
                <input type="text">
            </form>
            <div class="close"></div>
            <ul id="results_list">
```

zobaczyłem kod bez okienka z logowaniem. Teraz bierzmy się za ciasteczk i JOSN POST.
Wkliełm poniższy kod na podstawie zebranych informacji.
curl -X POST -d '{"search":"flag"}' -b 'PHPSESSID=r4lrt1qfaef0jo0hd2mi34ekr1' -H 'Content-Type: application/json' http://154.57.164.82:30500/search.php
Po pierwsze zapytania są wysyłąne za pomocą metody post:



następnie wpisujemy id sesji php i link do strony na której ma się wszystko odbyć. Ona symuluje to co robi user tylko za pomocą komend w terminalu i pozyskuje informacje za pomocą odpowiedzi.
Dzięki temu wiem jak są wysyłane zapytania i odpowiedzi. Źle zabezpieczona strona może być groźna przy użyciu narzędzi w przeglądarce i wysyłaniu odpowiednio spreparowanych komend,

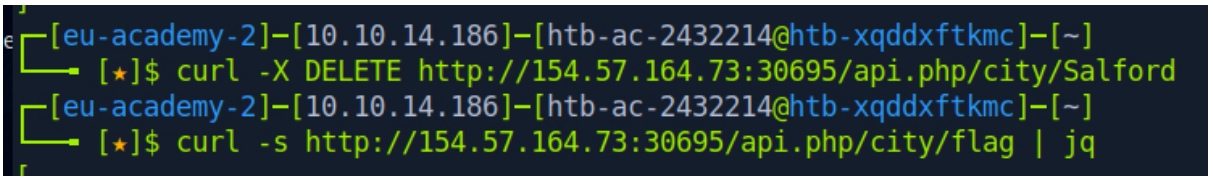
> **`-d '{"search":"flag"}'`**: Mówi serwerowi, **co** chcesz zrobić (przeszukaj w poszukiwaniu "flag").
> **`-b 'PHPSESSID=...'`**: Udowadnia serwerowi, **kim** jesteś (uwierzytelnionym użytkownikiem z aktywną sesją).

`curl -X POST http://154.57.164.73:30695/api.php/city/ -d '{"city_name":"flag", "country_name":"(UK)"}' -H 'Content-Type: application/json'` - dodanie rekordu do zapisu

`curl -X POST http://154.57.164.65:32381/api.php/city/ -d '{"flag":"HTB_City", "country_name":"HTB"}' -H 'Content-Type: application/json'` - to wkleiłem i dodałem

`curl -s http://154.57.164.65:32381/api.php/city/HTB_City | jq` - to jest do sprawdzenia o co chodzi

`curl -X DELETE http://<SERVER_IP>:<PORT>/api.php/city/New_HTB_City` - do usuniecia htb city
`curl -X DELETE http://154.57.164.65:32381/api.php/city/New_HTB_City`



w zadaniu trzeba było dodać usunąć i odczytać. Poszukam na medium i to mi pomogło. Trzeba było usunąć najstarsza na liście mieście na samym dole tylko że kraju uk i dopiero wtedy zadziałało.